MMAR 是一个考察audio reasoning能力的benchmark，有1000个任务。

一、任务格式

每个任务的格式如下：

```
@dataclass
class EachItem:
    id : str # unique id
    audio_path : str # the local path of the audio file
    question : str # question of this task
    choices : list[str] # the answer pool
    answer : str # the right answer
    thinking : str # the cot of how to get the right answer
    modality : str # sound(16.5%),speech(29.4%),music(20.6%),mix-sound-speech(21.8%),mix-
sound-music(1.1%),mix-speech-music(8.2%),mix-sound-speech-music(2.4%)
    category : str # Signal Layer(4.3%),Perception Layer(40.4%),Semantic
Layer(41.2%),Cultural Layer(14.1%)
    sub-category : str # the value of each layer,see the below sub_categories dict
    language : str|None # the language contained in audio or null
    source : str # youtube or something
    url : str # the website url of the origin video
    timestamp : str # the begining of the origin video to the end of the origin video
    cue : list[str] # the key to answer
    rubric : list[Rubric] # the standard of llm judger

@dataclass
class Rubric:
    name : str
    scoring_point : str
    note : str
```

```
    choices : list[str]

sub_categories : dict[str,list[str]] = {
    "Signal Layer":[
        "Acoustic Quality Analysis",
        "Anomaly Detection",
        "Audio Difference Analysis"
    ],
    "Perception Layer":[
        "Spatial Analysis",
        "Temporal Analysis",
        "Correlation Analysis",
        "Counting and Statistics",
        "Music Theory",
        "Environmental Perception and Reasoning"
    ],
    "Semantic Layer":[
        "Content Analysis",
        "Emotion and Intention",
        "Speaker Analysis"
    ],
    "Cultural Layer":[
        "Culture of Speaker",
        "Imagination",
        "Aesthetic Analysis"
    ]
}
```

二、评测方式

可以从`https://github.com/ddlBoJack/MMAR`仓库提供的`MMAR/MMAR-meta.jsonl`获取数据集文件，其code/文件夹下还提供了评测脚本（string_match和llm_as_a_judger两种方式）。

2.1 string_match

其中string_match的实现方法非常值得学习：

```
def string_match(answer, prediction, choices):
    # Function to normalize and tokenize text
    def tokenize(text):
        # Convert to lowercase and find all word tokens
        return set(re.findall(r'\b\w+\b', text.lower()))

    # Tokenize prediction and answer
    prediction_tokens = tokenize(prediction)
    answer_tokens = tokenize(answer)

    if not prediction_tokens:
        return False

    # Tokenize incorrect choices and exclude tokens present in the answer
    incorrect_tokens = set()
```

```
    for choice in choices:
        choice_tokens = tokenize(choice)
        if choice_tokens != answer_tokens:
            incorrect_tokens.update(choice_tokens - answer_tokens)

    # Condition 1: All tokens of the answer are in the prediction
    cond1 = answer_tokens.issubset(prediction_tokens)

    # Condition 2: Prediction does not contain any tokens from incorrect choices (excluding
shared words)
    cond2 = prediction_tokens.isdisjoint(incorrect_tokens)

    return cond1 and cond2
```

2.2 llm as a judger

llm as a judger的精髓是，最好设置明确的rubrics来帮助llm客观的评分，多次评分（比如5次），去掉最高最低分后取平均分。

三、评测对象

评测对象有:

- LALMs
- LARMs
- OLMs
- LLMs(with audio captions,so it's a cascaded system)
- LRMs(with audio captions,so it's a cascaded system)

四、实验设计和结果

4.1

4.1.1 不同评测对象在MMAR上的跑分

Table 2: MMAR results across five model categories: LALMs, LARMs, OLMs, LLMs, and LRMs with audio captions as input. The best-performing models in each category are highlighted in **bold**, and the second-best ones are underlined.

Models	Size	Single Modality (%)			Mixed Modalities (%)				Avg (%)
		Sound	Music	Speech	Sound-Music	Sound-Speech	Music-Speech	Sound-Music-Speech	
Random Guess	-	29.39	25.88	31.48	25.00	29.30	31.10	28.13	29.32
Large Audio Language Models (LALMs)									
Audio Flamingo [17]	2.2B	32.73	21.84	24.83	18.18	30.28	24.39	25.00	26.60
Audio Flamingo 2 [9]	0.5B	20.61	20.39	24.15	27.27	23.85	26.83	25.00	23.00
Audio Flamingo 2 [9]	1.5B	26.67	20.87	22.79	9.09	22.94	23.17	20.83	22.90
Audio Flamingo 2 [9]	3B	24.85	17.48	20.75	18.18	26.61	23.17	8.33	21.90
LTU [11]	7B	19.39	19.90	13.95	18.18	24.77	21.95	16.67	19.20
LTU-AS [10]	7B	20.00	14.08	19.05	9.09	20.64	28.05	12.50	19.00
MusiLingo [4]	7B	9.09	7.28	4.08	9.09	6.88	7.32	8.33	6.60
MU-LLaMA [23]	7B	13.94	13.59	14.97	9.09	12.39	14.63	16.67	13.90
GAMA [7]	7B	29.09	24.27	27.89	27.27	24.77	28.05	20.83	26.50
GAMA-IT [7]	7B	22.42	16.02	12.24	36.36	22.48	14.63	12.50	17.40
Qwen-Audio-Chat [2]	8.4B	27.88	20.39	22.11	9.09	25.23	25.61	20.83	23.50
Qwen2-Audio [3]	8.4B	33.94	23.30	32.99	9.09	33.03	26.83	33.33	30.40
Qwen2-Audio-Instruct [3]	8.4B	33.33	24.27	32.31	9.09	31.19	30.49	25.00	30.00
SALMONN [30]	7B	30.91	29.61	34.35	9.09	37.61	28.05	37.50	32.80
SALMONN [30]	13B	30.30	31.07	34.69	9.09	34.86	35.37	41.67	33.20
GPT-4o mini Audio [15]	-	38.79	35.92	58.84	45.45	60.09	57.32	50.00	50.60
GPT-4o Audio [15]	-	53.94	**50.97**	70.41	63.64	**72.48**	62.20	**75.00**	63.50
Large Audio Reasoning Models (LARMs)									
Mellow [5]	167M	33.33	26.70	24.83	18.18	37.16	32.93	29.17	30.00
Audio-CoT [25]	8.4B	35.76	25.24	34.01	9.09	30.73	30.49	37.50	31.30
Audio-Reasoner [35]	8.4B	43.64	33.50	32.99	45.45	42.66	31.71	25.00	36.80
Omni Language Models (OLMs)									
AnyGPT-chat [39]	8B	24.24	19.42	22.11	27.27	27.52	26.83	29.17	23.70
OpenOmni [24]	8B	20.61	22.33	35.37	18.18	27.06	23.17	25.00	27.00
Baichuan-Omni-1.5 [19]	11B	41.21	33.01	40.48	36.36	48.62	39.02	41.67	40.70
Qwen-2.5-Omni [36]	3B	53.94	46.12	53.74	36.36	60.09	57.32	58.33	53.80
Qwen-2.5-Omni [36]	7B	58.79	40.78	59.86	54.55	61.93	**67.07**	58.33	56.70
Gemini 2.0 Flash [12]	-	**61.21**	**50.97**	**72.11**	**81.82**	**72.48**	65.85	70.83	**65.60**
Large Language Models (LLMs)									
Caption + DeepSeek-V3 [21]	671B	42.42	40.78	56.12	18.18	50.00	45.12	37.50	47.60
Caption + GPT-4o [15]	-	46.06	40.29	60.88	27.27	53.67	46.34	45.83	50.70
Large Reasoning Models (LRMs)									
Caption + DeepSeek-R1 [13]	671B	46.67	49.51	62.59	45.45	58.72	56.10	54.17	55.50
Caption + OpenAI o1 [16]	-	48.48	43.20	63.61	18.18	56.88	45.12	45.83	53.00
Caption + OpenAI o3 [28]	-	49.70	41.75	63.95	36.36	60.09	52.44	54.17	54.70

4.1.2 poisson binomial distribution+bonferroni证明大部分开源模型几乎等于瞎蒙

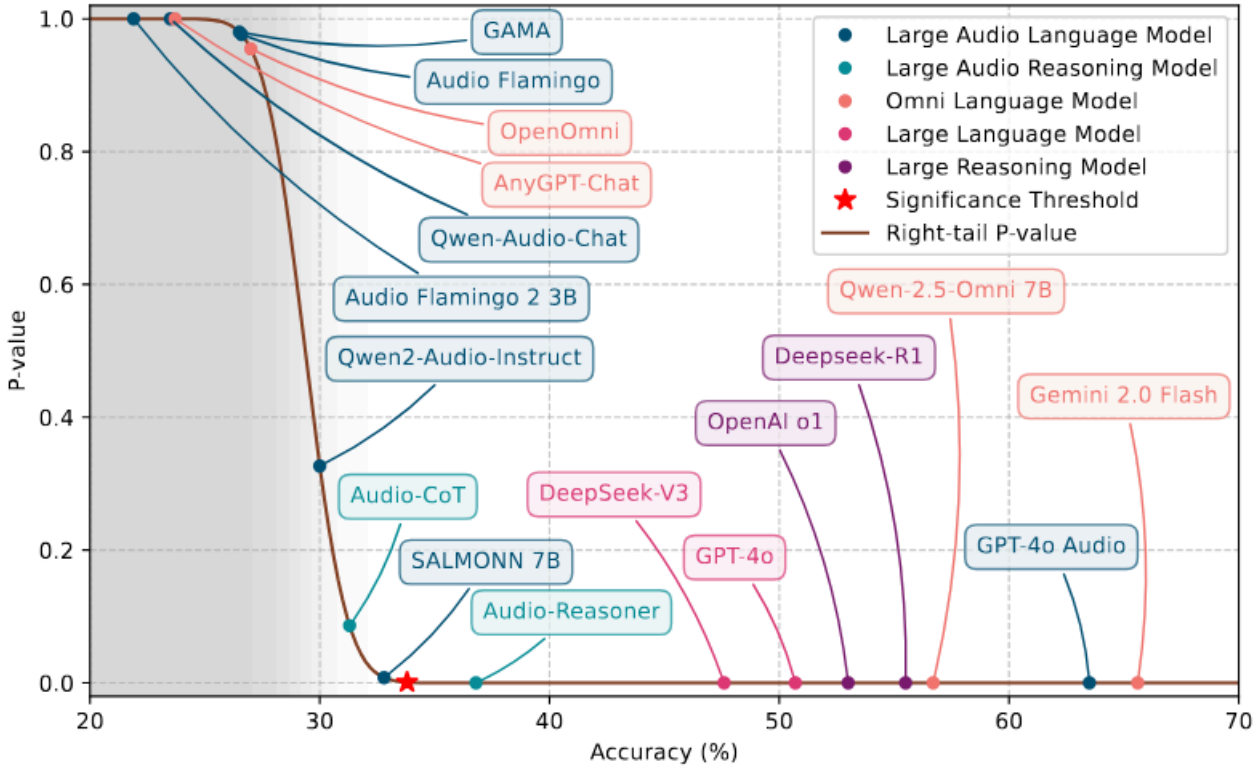


Figure 4: Poisson Binomial distribution illustrating the P-Value of accuracy under random guessing. For LLMs and LRMs, audio captions are provided. One-tailed right-sided P-value is computed to test whether each model significantly outperforms random guessing. Solid circles mark the observed performance of each model. A red star indicates the Bonferroni-corrected significance threshold ($\alpha = 0.001$).

4.1.3 MMAR和MMAU的准确率对比实验

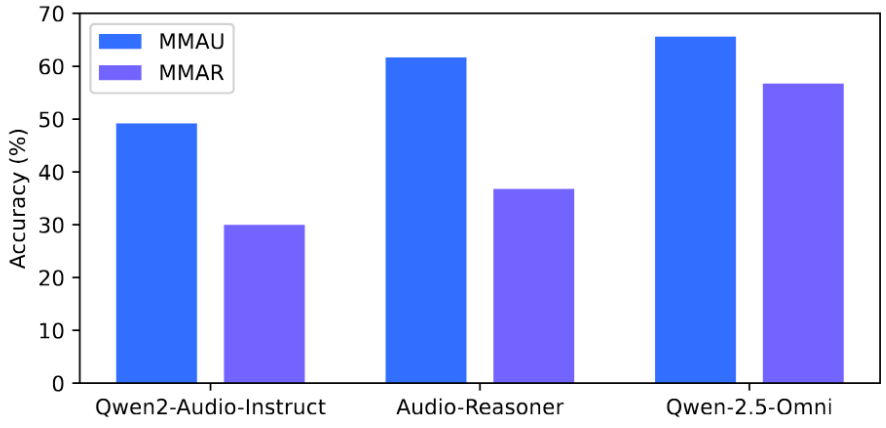


Figure 8: Different Audio-Language models' performance on MMAU (test-mini) and MMAR.

4.1.4 结论

- MMAR很难
- 开源模型距离闭源还有很大的差距
- 无论什么模型和系统，cot对准确率的加成明显，说明了其必要性

4.1.5 数学知识补贴

1.poisson binomial：

设定 N道题（MMAR约1000道），第 i 道题瞎蒙对的概率记为 p_i（四选一就大约是0.25，逐题可微调），H_0假设："这模型纯瞎蒙"。

第一步：算"瞎蒙总对数"的完整分布

定义随机变量 S = 瞎蒙情况下总共蒙对的题数。S是N个独立伯努利变量的和——每个p_i不同时，S的分布就叫Poisson Binomial。

它的概率质量函数用递推算：

$$P_n(k) = p_n \cdot P_{n-1}(k-1) + (1-p_n) \cdot P_{n-1}(k)$$

$$P_1(0) = 1 - p_1$$

$$P_1(1) = p_1$$

p(a) = P(S ≥ a·N) = 瞎蒙蒙对a以上的概率 = 把分布曲线右侧尾巴的面积加起来，可以画出p-a图，如figure4所示。

论文是按照每个题$p_i = 1/choices$画的曲线图。

2.Bonferroni：

我们设置每个模型p < 0.001就是判决线的位置：

对每个模型，我们都算出它的p值（纯瞎蒙蒙出它这个成绩的概率）。然后立一条规矩：只有当你的p值小于0.001，我才承认你"显著强于瞎蒙"；p ≥ 0.001，对不起，你的成绩手气就能解释，不予采信。

问题：这个阈值下，30个模型的总冤枉率是多少？

单个检验不冤枉的概率 = 1 − 0.001 = 0.999

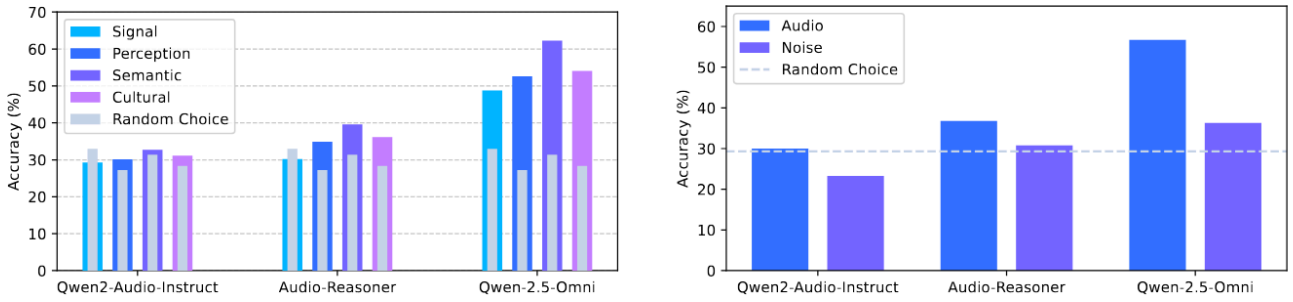
30个检验相互独立，全都不冤枉 = 30个0.999连乘：$(1 - 0.001)^{30} \approx 0.97$

至少冤枉一个的概率：1 − 0.97 ≈ 0.03

结论：总冤枉率3%，比5%的要求还严格，合格。所以0.001这个阈值站得住。

4.2 对比实验（消融实验）

4.2.1 不同layer的准确率对比实验，和random choice的对比实验，和noise input的对比实验：



(a) Compare different reasoning hierarchies on MMAR (b) Compare with noise input on MMAR

Figure 5: Performance comparison on the MMAR benchmark. **(a)** compares task accuracy on different reasoning hierarchies, and **(b)** shows the impact of using noise as input rather than audio.

4.2.2 级联audio captions+llm/lrm的系统，不同基座的对比实验：

Table 3: Compare audio caption models and LLMs in cascaded models.

Reasoning\Perception	Qwen2-Audio -Instruct	Qwen-2.5 -Omni
GPT-4o	50.70	51.80
OpenAI o1	53.00	54.40

4.2.3 结论

- signal layer对模型最难，semantic layer对模型最简单
- MMAR的局限性：即使流程规范，仍然有一定文本先验的干扰
- 提升audio captions的质量和llm/lrm两个部分都有助于MMAR指标的提升

4.3 错误分析实验

错误归因：

```
error_type : dict[str,int] ={
    "Perceptual Error": 0.37,
    "Knowledger Error": 0.09,
    "Reasoning Error": 0.20,
    "Other Error": 0.34,
}

OtherError_type : dict[str,int] ={
    "Wrong Understanding of Instruction":0.03,
    "Choice Format Error":0.04,
    "Repeat Thinking and No Answer":0.12,
    "Correct Answer but Wrong Choice":0.15,
}
```

五、总结

MMAR 是一个考察audio reasoning能力的benchmark，有1000个任务，每个任务有固定的格式，支持两种评测方式(string_match和llm_as_a_judger)，两种的实现方式都非常值得学习。
MMAR 通过一系列实验，证明了以下结论：

- MMAR很难
- 开源模型距离闭源还有很大的差距,大部分开源模型在MMAR上和瞎猜没差别
- 无论什么模型和系统，cot对准确率的加成明显，说明了其必要性
- signal layer对模型最难，semantic layer对模型最简单
- MMAR的局限性：即使流程规范，仍然有一定文本先验的干扰
- 提升audio captions的质量和llm/lrm两个部分都有助于MMAR指标的提升